

Red-Black Tree Validation

Red-Black Trees, Invariants, Validation · Mentor-Led Lab (Student Edition)
Developed by TalenciaGlobal

Difficulty ★★★ Intermediate

Problem

Pattern: tree-property validation. Given a colored binary tree, decide whether it is a valid red-black tree and report its black-height.

A valid red-black tree is search-ordered, has a black root, never has a red node with a red child, and crosses the same number of black nodes on every root-to-leaf path.

What to Compute

- Valid: Yes if all red-black invariants hold, otherwise No.
- Black Height: the number of black nodes on a root-to-NIL path (excluding NIL) when valid, otherwise -1.

Input / Output Format

Input: One line: the tree in level order; each token is key:color (R or B), with N for a missing child.

Output: Two lines: Valid and Black Height.

Examples

Example 1

Input: 10:B 5:R 15:R 3:B 7:B 12:B 18:B

Output:

Valid: Yes

Black Height: 2

Explanation: Search-ordered, black root, the red nodes 5 and 15 have black children, and every path crosses two black nodes - a valid red-black tree.

Example 2

Input: 10:B 5:R 15:R 4:R N N N

Output:

Valid: No

Black Height: -1

Explanation: The red node 5 has a red child 4, violating the no-red-red rule, so the tree is invalid.

Constraints

- The root, if present, must be black.
- No red node may have a red child.
- All root-to-NIL paths must have equal black-height, and keys must obey the BST property.

Sample Run

Sample Input

```
10:B 5:R 15:R 3:B 7:B 12:B 18:B
```

Sample Output

```
Valid: Yes  
Black Height: 2
```